imging

Input.tif → .npy → .npy → .png

train_loop():

data load() → model() → forward() → pred/Loss() → Backward → optimizer() → checking-point() for long time traing.

optimizer(): adjust weights

checking-point(): when crashed, checking-point() can recover/reload ~~M~~ Models/states efficiently

---

precision goes low ⇒ traing stage becomes numerically unstable.
e.g. FP8.

Mixed_precision_training()

⇓

~~H~~ Hardware sys ~~desig~~ design and model architecture design

are synergistic

⇓

e.g. Transformer trained on Nvidia GPU with ~~FP~~ ~~LF8~~ FP8 would be faster than models trained on ~~other~~ the same GPU with FP32.

Lecture #3: Modern LLM architecture.

original Transformer

⇓

variants:

⇓

architecture

- norm layer in the front of block (pre-norm/post-norm)
- RoPE (position embedings)
- FF layers use SwiGLU, not ReLU
- Attention variants

#1: ~~post~~ post-norm versus ~~post~~ pre-norm << versus: double norm

post-norm: x_i ⇒ attention ⇒ addition ⇒ norm ⇒ FFN ⇒ add ⇒ norm

⇒

pre-norm: x_i ⇒ norm. ⇒ attention ⇒ addition ⇒ norm ⇒ FFN ⇒ add

<<

double norm: X → Layer Norm → multi-head attention → norm → addition → norm layer → FFN → norm layer → add

#2: LayerNorm versus RMSNorm

LayerNorm ⇓ normailze mean and variance across dmodel ⇓ previous: GPT-3/2/1

RMSNorm ⇓ does not substract mean or add a bias term ⇓ LLama/T5: faster/fewer operations